

creagia / laravel-sign-pad

Code

Issues

Pull requests3

Discussions

Actions

Projects

Security

Insights

laravel-sign-padPublic

main

3 Branches

10 Tags

Go to file

Go to file

+

about

Code

...

github-actions[bot] Me... fa9e608 · 8 months ago 41 Commits

.github	build(deps): bump dependa...	8 months ago
config	Add upgrading information	10 months ago
database/migrations	First commit	2 years ago
resources	added disabled without sig...	8 months ago
routes	First commit	2 years ago
src	added disabled without sig...	8 months ago
tests	Configurable disk option	10 months ago
.gitignore	First commit	2 years ago
LICENSE.md	fix: License for Packagist	2 years ago
README.md	Update readme	8 months ago
UPGRADING.md	Add upgrading information	10 months ago
composer.json	Laravel 10 compatibility (#2...	last year
package-lock.json	First commit	2 years ago
package.json	First commit	2 years ago
phpstan.neon.dist	First commit	2 years ago
phpunit.xml.dist	First commit	2 years ago
webpack.config.js	First commit	2 years ago

A Laravel package to sign documents and optionally generate certified PDFs associated with Eloquent models.

#pdf #laravel #signature #signature-pad

Readme

MIT license

Security policy

Activity

Custom properties

414 stars

7 watching

35 forks

Report repository

Releases10

2.1.0Lateston Jul 3, 2023

+ 9 releases

Sponsor this project

Packages

No packages published

Contributors10

READMEMIT licenseSecurity

Laravel pad signature

A Laravel package to sign documents and optionally generate [certified PDFs](#) associated to a Eloquent model

packagistv2.1.0

testspassing

downloads10k

Languages

PHP90.5%

JavaScript5.9%

Blade3.6%



A modern approach to hire Laravel developers

Get hired or directly hire the best Laravel developers.

Create your profile now and get discovered by companies hiring developers.

Requirements

Laravel pad signature requires **PHP 8.0, 8.1 or 8.2** and **Laravel 8, 9 or 10**.

Installation

You can install the package via composer:

```
composer require creagia/laravel-sign-pad
```



Publish the config and the migration files and migrate the database

```
php artisan sign-pad:install
```



Publish the .js assets:

```
php artisan vendor:publish --tag=sign-pad-assets
```



This will copy the package assets inside the `public/vendor/sign-pad/` folder.

Configuration

In the published config file `config/sign-pad.php` you'll be able to configure many important aspects of the package, like the route name where users will be redirected after signing the document or where do you want to store the signed documents. You can customize the disk and route to store signatures and documents.

Notice that the `redirect_route_name` will receive the parameter `$uuid` with the uuid of the signature model in the database.

Preparing your model

Add the `RequiresSignature` trait and implement the `CanBeSigned` class to the model you would like.

```
<?php

namespace App\Models;

use Creagia\LaravelSignPad\Concerns\RequiresSignature;
use Creagia\LaravelSignPad\Contracts\CanBeSigned;

class MyModel extends Model implements CanBeSigned
{
    use RequiresSignature;
```



```
}
```

```
?>
```

If you want to generate PDF documents with the signature, you should implement the `ShouldGenerateSignatureDocument` class. Define your document template with the `getSignatureDocumentTemplate` method.

```
<?php
```

```
namespace App\Models;
```

```
use Creagia\LaravelSignPad\Concerns\RequiresSignature;
```

```
use Creagia\LaravelSignPad\Contracts\CanBeSigned;
```

```
use Creagia\LaravelSignPad\Contracts\ShouldGenerateSignatureDocument;
```

```
use Creagia\LaravelSignPad\Templates\BladeDocumentTemplate;
```

```
use Creagia\LaravelSignPad\Templates\PdfDocumentTemplate;
```

```
use Creagia\LaravelSignPad\SignatureDocumentTemplate;
```

```
use Creagia\LaravelSignPad\SignaturePosition;
```

```
class MyModel extends Model implements CanBeSigned, ShouldGenerateSignatureDocument
```

```
{
```

```
    use RequiresSignature;
```

```
    public function getSignatureDocumentTemplate(): SignatureDocumentTemplate
```

```
    {
```

```
        return new SignatureDocumentTemplate(
```

```
            outputPdfPrefix: 'document', // optional
```

```
            // template: new BladeDocumentTemplate('pdf/my-pdf-blade-template'), // Uncomment for Blade template
```

```
            // template: new PdfDocumentTemplate(storage_path('pdf/template.pdf')), // Uncomment for PDF template
```

```
            signaturePositions: [
```

```
                new SignaturePosition(
```

```
                    signaturePage: 1,
```

```
                    signatureX: 20,
```

```
                    signatureY: 25,
```

```
                ),
```

```
                new SignaturePosition(
```

```
                    signaturePage: 2,
```

```
                    signatureX: 25,
```

```
                    signatureY: 50,
```

```
                ),
```

```
            ]
```

```
        );
```

```
    }
```

```
}
```

```
?>
```

A `$model` object will be automatically injected into the Blade template, so you will be able to access all the needed properties of the model.

Usage

At this point, all you need is to create the form with the sign pad canvas in your template. For the route of the form, you have to call the method `getSignatureUrl()` from the instance of the model you prepared before:

```
@if (!$myModel->hasBeenSigned())
```

```
    <form action="{ { $myModel->getSignatureRoute() } }" method="POST">
```

```
        @csrf
```

```
        <div style="text-align: center">
```

```
            <x-creagia-signature-pad />
```

```
        </div>
```

```
</form>
<script src="{{ asset('vendor/sign-pad/sign-pad.min.js') }}"></script>
@endif
```

Retrieving signatures

You can retrieve your model signature using the Eloquent relation `$myModel->signature` . After that, you can use:

- `getSignatureImagePath()` returns the signature image path.
- `getSignatureImageAbsolutePath()` returns the signature image absolute path.
- `getSignedDocumentPath()` returns the generated PDF document path.
- `getSignedDocumentAbsolutePath()` returns the generated PDF document absolute path.

```
echo $myModel->signature->getSignatureImagePath();
echo $myModel->signature->getSignedDocumentPath();
```



Deleting signatures

You can delete your model signature using

- `deleteSignature()` method in the model.

```
echo $myModel->deleteSignature();
```



Customizing the component

From the same template, you can change the look of the component by passing some properties:

- *border-color* (hex) to change the border color of the canvas
- *pad-classes* and *button-classes* (strings) indicates which classes will have the sign area or the submit & clear buttons
- *clear-name* and *submit-name* (strings) allows you to modify the default "Submit" and "Clear" values of the buttons.
- *disabled-without-signature* (boolean) indicates if the submit button should be disabled when the user has not signed yet.

An example with an app using Tailwind would be:

```
<x-creagia-signature-pad
  border-color="#eaeaea"
  pad-classes="rounded-xl border-2"
  button-classes="bg-gray-100 px-4 py-2 rounded-xl mt-4"
  clear-name="Clear"
  submit-name="Submit"
  :disabled-without-signature="true"
/>
```



Certifying the PDFs

To certify your signature with TCPDF, you will have to create your own SSL certificate with OpenSSL. Otherwise you can find the TCPDF demo certificate here : [TCPDF Demo Certificate](#)

To create your own certificate use this command :

```
cd storage/app
openssl req -x509 -nodes -days 365000 -newkey rsa:1024 -keyout certificate.crt -out certificate.crt
```



More information in the [TCPDF documentation](#)

After generating the certificate, you'll have to change the value of the variable `certify_documents` in the `config/sign-pad.php` file and set it to **true**.

When the variable `certify_documents` is set to true, the package will search the file allocated in the `certificate_file` path to sign the documents. Feel free to modify the location or the name of certificate file by changing its value.

Inside the same `config/sign-pad.php` we encourage you to fill all the fields of the array `certificate_info` to be more specific with the certificate.

Finally, you can change the certificate type by modifying the value of the variable `cert_type` (by default 2). You can find more information about certificates types at [TCPDF setSignature reference](#).

Upgrading

See [UPGRADING](#) for details.

License